100 Database Developer Interview Questions & Answers

MSSQL & MySQL Focus

Prepared for a candidate with approximately 1.3 years of experience
Covers SQL fundamentals through practical, scenario-based questions

Target Role: Database Developer | Company: NeoSoft

Table of Contents

1. SQL Fundamentals & DDL/DML (Q1–Q9)
2. Joins & Set Operations (Q10–Q17)
3. Keys & Constraints (Q18–Q25)
4. Indexes & Performance (Q26–Q33)
5. Stored Procedures, Functions & Views (Q34–Q41)
6. Triggers (Q42–Q47)
7. Transactions & Concurrency (Q48–Q54)
8. Normalization & Database Design (Q55–Q61)
9. Advanced Queries & Window Functions (Q62–Q68)
10. MSSQL vs MySQL – Platform-Specific Concepts (Q69–Q76)
11. Backup, Recovery, Security & Administration (Q77–Q82)
12. Performance Tuning & Query Optimization (Q83–Q88)
13. Error Handling & Programming Constructs (Q89–Q93)
14. Scenario-Based & Practical Questions (Q94–Q97)
15. Role & Experience-Based Questions (Q98–Q100)

1. SQL Fundamentals & DDL/DML

Q1. What is SQL and what are its main sub-languages?

Answer: SQL (Structured Query Language) is used to create, manage, and query relational databases. Its main sub-languages are: DDL (Data Definition Language - CREATE, ALTER, DROP), DML (Data Manipulation Language - SELECT, INSERT, UPDATE, DELETE), DCL (Data Control Language - GRANT, REVOKE), and TCL (Transaction Control Language - COMMIT, ROLLBACK, SAVEPOINT).

Q2. What is the difference between DELETE, TRUNCATE, and DROP?

Answer: DELETE removes rows one at a time, is DML, can use WHERE, is logged (can be rolled back) and fires triggers. TRUNCATE removes all rows at once, is DDL, cannot use WHERE, is minimally logged and resets identity, and is faster. DROP removes the entire table structure along with its data from the database.

Q3. What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before grouping/aggregation and cannot use aggregate functions. HAVING filters groups after GROUP BY and is used with aggregate functions like SUM(), COUNT(), AVG().

Q4. What is the difference between CHAR and VARCHAR data types?

Answer: CHAR is a fixed-length data type that pads with spaces to the defined length, making it faster for fixed-size data. VARCHAR is variable-length, stores only the actual characters plus a small length indicator, and saves storage space for variable-size data.

Q5. What is the difference between UNION and UNION ALL?

Answer: UNION combines result sets from two or more SELECT statements and removes duplicate rows, which requires an internal sort/distinct operation making it slower. UNION ALL combines result sets and keeps all duplicates, making it faster since no de-duplication is performed.

Q6. What is a NULL value and how is it different from zero or blank?

Answer: NULL represents an unknown or missing value, not zero or empty string. NULL cannot be compared using = or != operators; instead IS NULL or IS NOT NULL must be used. Any arithmetic or string operation involving NULL typically returns NULL.

Q7. What is the difference between INSERT, UPDATE, and MERGE (UPSERT)?

Answer: INSERT adds new rows to a table. UPDATE modifies existing rows based on a condition. MERGE (available in MSSQL, and via INSERT ... ON DUPLICATE KEY UPDATE in MySQL) combines both - it inserts a row if it does not exist and updates it if it does, based on a matching condition.

Q8. What are the different types of SQL statements based on execution order?

Answer: The logical processing order is: FROM, JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, and finally LIMIT/OFFSET (or TOP/OFFSET-FETCH). Understanding this order helps explain why column aliases from SELECT cannot be used in WHERE.

Q9. What is the difference between a schema and a database?

Answer: A database is a collection of related data, tables, views, and other objects. A schema is a logical namespace or container within a database used to group and organize objects (like dbo in MSSQL). MySQL treats 'schema' and 'database' as synonyms, while MSSQL treats schema as a separate logical layer inside a database.

2. Joins & Set Operations

Q10. What are the different types of JOINs in SQL?

Answer: INNER JOIN returns matching rows from both tables. LEFT (OUTER) JOIN returns all rows from the left table plus matches from the right (NULL if no match). RIGHT (OUTER) JOIN is the reverse. FULL OUTER JOIN returns all rows from both tables. CROSS JOIN returns the Cartesian product. SELF JOIN joins a table to itself.

Q11. What is a self join and when would you use it?

Answer: A self join is when a table is joined with itself using table aliases. It is commonly used for hierarchical data, such as finding an employee's manager from the same Employees table by joining EmployeeID to ManagerID.

Q12. Does MySQL support FULL OUTER JOIN?

Answer: MySQL does not directly support FULL OUTER JOIN. It is simulated using a UNION of a LEFT JOIN and a RIGHT JOIN (or UNION ALL with a NOT EXISTS check to avoid duplicate matching rows). MSSQL supports FULL OUTER JOIN natively.

Q13. What is the difference between INNER JOIN and CROSS JOIN?

Answer: INNER JOIN returns only rows where a specified join condition matches between tables. CROSS JOIN returns the Cartesian product - every row of the first table combined with every row of the second table, with no join condition.

Q14. How do you find duplicate records in a table using SQL?

Answer: Group by the column(s) that should be unique and use HAVING COUNT(*) > 1. For example: SELECT email, COUNT(*) FROM Users GROUP BY email HAVING COUNT(*) > 1. To retrieve full duplicate rows, this can be combined with a JOIN or a window function like ROW_NUMBER().

Q15. How do you write a query to find records in one table that don't exist in another?

Answer: Use LEFT JOIN with a WHERE clause checking for NULL on the right table's key, or use NOT EXISTS / NOT IN. NOT EXISTS is generally preferred for performance and correct NULL handling: SELECT a.* FROM TableA a WHERE NOT EXISTS (SELECT 1 FROM TableB b WHERE b.id = a.id).

Q16. What is the difference between EXISTS and IN?

Answer: IN compares a value against a list or subquery result and can behave unexpectedly with NULLs. EXISTS checks for the existence of rows returned by a correlated subquery and stops at the first match, which is often more efficient, especially on large datasets, and handles NULLs more predictably.

Q17. How would you combine results from two tables with the same structure without duplicates?

Answer: Use UNION, which automatically removes duplicate rows between the two result sets, as opposed to UNION ALL which keeps all rows including duplicates.

3. Keys & Constraints

Q18. What is a Primary Key and what are its properties?

Answer: A Primary Key uniquely identifies each row in a table. It must contain unique values, cannot contain NULL, and a table can have only one primary key (though it can be composite, spanning multiple columns). It automatically creates a unique clustered index in MSSQL by default.

Q19. What is a Foreign Key and why is it used?

Answer: A Foreign Key is a column (or set of columns) that references the Primary Key of another table, establishing a relationship between the two tables and enforcing referential integrity so that you cannot insert a value that doesn't exist in the referenced table.

Q20. What is the difference between Primary Key and Unique Key?

Answer: A Primary Key does not allow NULL values and only one can exist per table. A Unique Key allows one NULL value (in most RDBMS) and a table can have multiple unique keys. Both enforce uniqueness of values.

Q21. What is a Composite Key?

Answer: A Composite Key is a primary key made up of two or more columns that together uniquely identify a row, used when no single column is sufficient to guarantee uniqueness, such as an OrderID + ProductID combination in an OrderDetails table.

Q22. What is a Candidate Key?

Answer: A Candidate Key is any column or combination of columns that can qualify as a unique identifier for rows in a table. A table may have multiple candidate keys, and one of them is chosen as the Primary Key.

Q23. What are the different types of constraints in SQL?

Answer: NOT NULL ensures a column cannot have NULL values. UNIQUE ensures all values are distinct. PRIMARY KEY uniquely identifies rows. FOREIGN KEY enforces referential integrity. CHECK ensures values satisfy a condition. DEFAULT provides a default value when none is specified.

Q24. What happens when you delete a row referenced by a Foreign Key?

Answer: It depends on the referential action defined: RESTRICT/NO ACTION blocks the delete if references exist, CASCADE deletes the dependent rows automatically, SET NULL sets the foreign key column to NULL, and SET DEFAULT sets it to a default value.

Q25. Can a table have multiple Foreign Keys?

Answer: Yes, a table can have multiple foreign keys, each referencing the primary key (or a unique key) of a different table, allowing it to participate in multiple relationships.

4. Indexes & Performance

Q26. What is an index and why is it used?

Answer: An index is a database object that improves the speed of data retrieval by creating a sorted structure (typically a B-Tree) on one or more columns, at the cost of additional storage and slower write operations since the index must also be updated.

Q27. What is the difference between Clustered and Non-Clustered Index?

Answer: A Clustered Index determines the physical storage order of data in the table, so a table can have only one. A Non-Clustered Index is a separate structure that stores pointers back to the actual data rows, and a table can have multiple non-clustered indexes.

Q28. Does MySQL (InnoDB) support clustered indexes the same way as MSSQL?

Answer: In InnoDB (MySQL's default engine), the table data is always physically organized by the primary key, which acts like a clustered index automatically. If no primary key exists, InnoDB uses a hidden internal row ID. This is similar in concept to MSSQL's clustered index but is tied directly to the primary key.

Q29. What is a covering index?

Answer: A covering index includes all the columns needed to satisfy a query (in the index itself), so the database engine doesn't need to look up the actual table (avoiding a 'key lookup' or 'bookmark lookup'), which significantly improves query performance.

Q30. When should you avoid creating too many indexes?

Answer: Excessive indexes slow down INSERT, UPDATE, and DELETE operations since every index must be maintained on each write. They also consume extra disk space. Indexes should be created based on actual query patterns, not on every column.

Q31. What is index fragmentation and how do you fix it?

Answer: Fragmentation occurs when the logical order of index pages doesn't match physical storage order, often due to frequent inserts/updates/deletes, degrading performance. In MSSQL it's fixed using ALTER INDEX REORGANIZE (light) or REBUILD (heavy); in MySQL it's typically fixed with OPTIMIZE TABLE.

Q32. How do you check whether a query is using an index?

Answer: In MSSQL, use the Execution Plan (Ctrl+M) or SET STATISTICS IO ON to see logical reads. In MySQL, use EXPLAIN before the query to see whether it does a full table scan or uses an index, along with the 'key' and 'rows' columns in the output.

Q33. What is the difference between a unique index and a primary key constraint?

Answer: A primary key is both a constraint (enforcing entity integrity, disallowing NULLs) and it automatically creates a unique index. A unique index/constraint enforces uniqueness but can allow one NULL and a table can have several of them.

5. Stored Procedures, Functions & Views

Q34. What is a Stored Procedure and what are its advantages?

Answer: A Stored Procedure is a precompiled, reusable block of SQL code stored in the database that can accept parameters and perform operations. Advantages include improved performance (execution

plan caching), reduced network traffic, better security (users can be granted execute rights without table access), and reusability.

Q35. What is the difference between a Stored Procedure and a Function?

Answer: A Function must return a value and can be used within a SELECT statement, but cannot perform DML operations like INSERT/UPDATE/DELETE in most cases and cannot use TRY-CATCH in MSSQL. A Stored Procedure may or may not return a value, can perform DML/DDL, supports transactions and error handling, and cannot be called directly inside a SELECT statement.

Q36. How do you pass parameters to a Stored Procedure?

Answer: In MSSQL: CREATE PROCEDURE GetOrders @CustomerID INT AS SELECT * FROM Orders WHERE CustomerID = @CustomerID. In MySQL: CREATE PROCEDURE GetOrders(IN custId INT) BEGIN SELECT * FROM Orders WHERE CustomerID = custId; END. Both support IN, OUT, and INOUT parameter types.

Q37. What is a View and why would you use one?

Answer: A View is a virtual table based on the result of a SELECT query. It simplifies complex queries, provides an abstraction layer over the underlying tables, can restrict column/row access for security, and doesn't store data itself (except Indexed/Materialized views).

Q38. Can you perform DML operations (INSERT/UPDATE/DELETE) on a View?

Answer: Yes, but only if the view is 'updatable' - typically it must be based on a single table, must not contain aggregate functions, GROUP BY, DISTINCT, or set operations. Views based on multiple joined tables or aggregates are generally not directly updatable.

Q39. What is an Indexed View (MSSQL) / Materialized View concept?

Answer: An Indexed View in MSSQL physically stores the result set of a view by creating a unique clustered index on it, improving performance for complex, frequently-used aggregations at the cost of extra storage and maintenance overhead on writes. MySQL doesn't natively support materialized views, but they can be simulated using summary tables refreshed via triggers or scheduled events.

Q40. What is a User-Defined Function (UDF) and what types exist in MSSQL?

Answer: A UDF is custom logic that returns a value. MSSQL supports Scalar Functions (return a single value), Inline Table-Valued Functions (return a table, treated like a view), and Multi-statement Table-Valued Functions (build a table variable using multiple statements).

Q41. What is the difference between a temporary table and a table variable in MSSQL?

Answer: A temporary table (#temp) is stored in tempdb, supports indexes, constraints, and statistics, and is better for large datasets. A table variable (@table) is also stored in tempdb but has limited indexing options, no statistics (which can lead to poor query plans on large data), and its scope is limited to the batch/procedure it's declared in.

6. Triggers

Q42. What is a Trigger and what types are there?

Answer: A Trigger is special code that automatically executes in response to specific events (INSERT, UPDATE, DELETE) on a table. Types include AFTER (FOR) triggers, which run after the action, and

INSTEAD OF triggers, which run in place of the action (commonly used on views).

Q43. What are the INSERTED and DELETED tables in MSSQL triggers?

Answer: These are special logical tables available inside a trigger. INSERTED holds the new/affected rows for INSERT and UPDATE operations. DELETED holds the old rows for DELETE and UPDATE operations. For UPDATE, both tables are populated - DELETED has the old values and INSERTED has the new values.

Q44. How does MySQL handle old and new row values in triggers?

Answer: MySQL uses OLD and NEW keywords (not tables) to reference row values. OLD.column_name refers to the value before the change (available in UPDATE/DELETE), and NEW.column_name refers to the value after the change (available in INSERT/UPDATE).

Q45. What are common use cases for triggers?

Answer: Auditing/logging changes to sensitive tables, enforcing complex business rules that constraints can't handle, maintaining denormalized/summary data automatically, and preventing invalid operations (e.g., blocking deletes under certain conditions).

Q46. What are the disadvantages of using triggers?

Answer: Triggers can make debugging harder since logic executes implicitly/invisibly, they can degrade performance if overused or poorly written, they can create hidden cascading effects across other triggers, and they are harder to maintain and test compared to explicit application logic.

Q47. Can a trigger call another trigger? What is this called?

Answer: Yes, this is called a nested trigger. If TriggerA's action causes TriggerB to fire, that's nesting. Both MSSQL and MySQL support this to a limited depth, but excessive nesting can cause performance issues or infinite loops if not carefully controlled.

7. Transactions & Concurrency

Q48. What is a transaction and what are the ACID properties?

Answer: A transaction is a logical unit of work that must complete fully or not at all. ACID stands for Atomicity (all-or-nothing), Consistency (database moves from one valid state to another), Isolation (concurrent transactions don't interfere with each other), and Durability (once committed, changes persist even after a crash).

Q49. What is the difference between COMMIT and ROLLBACK?

Answer: COMMIT permanently saves all changes made during the current transaction to the database. ROLLBACK undoes all changes made during the current transaction, reverting the database to its state before the transaction began.

Q50. What are the different transaction isolation levels?

Answer: READ UNCOMMITTED (allows dirty reads), READ COMMITTED (default in MSSQL - no dirty reads), REPEATABLE READ (default in MySQL InnoDB - prevents non-repeatable reads), SERIALIZABLE (highest isolation, prevents phantom reads but reduces concurrency), and SNAPSHOT (MSSQL - uses row versioning).

Q51. What is a Deadlock and how can it be avoided?

Answer: A Deadlock occurs when two or more transactions block each other by each holding a lock the other needs, causing neither to proceed. It can be avoided by accessing tables/resources in a consistent order across transactions, keeping transactions short, using appropriate isolation levels, and adding indexes to reduce lock scope.

Q52. What is a lock and what types of locks exist in a database engine?

Answer: A lock is a mechanism used to control concurrent access to data and prevent conflicting operations. Common types include Shared (S) locks for reads, Exclusive (X) locks for writes, Update (U) locks used to avoid conversion deadlocks, and Intent locks that signal a lock is held at a lower level (row/page/table) of the hierarchy.

Q53. What is the difference between a dirty read, non-repeatable read, and phantom read?

Answer: A dirty read occurs when a transaction reads uncommitted data from another transaction. A non-repeatable read occurs when a row is read twice in the same transaction and the values differ because another transaction modified it in between. A phantom read occurs when a transaction re-runs a query and finds new rows that were inserted by another transaction.

Q54. What is optimistic vs pessimistic locking?

Answer: Pessimistic locking locks a resource as soon as a transaction wants to modify it, preventing other transactions from touching it until released. Optimistic locking allows multiple transactions to proceed without locking, and checks for conflicts (e.g., via a version/timestamp column) only at the time of commit.

8. Normalization & Database Design

Q55. What is Normalization and why is it important?

Answer: Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity by dividing large tables into smaller, related tables and defining relationships between them using foreign keys.

Q56. Explain 1NF, 2NF, and 3NF with simple examples.

Answer: 1NF (First Normal Form): Each column holds atomic (indivisible) values and each row is unique - no repeating groups or arrays in a single cell. 2NF: Table is in 1NF, and every non-key column is fully dependent on the entire primary key (removes partial dependency, relevant for composite keys). 3NF: Table is in 2NF, and no non-key column depends on another non-key column (removes transitive dependency).

Q57. What is Denormalization and when would you use it?

Answer: Denormalization is intentionally introducing redundancy into a normalized database, often by combining tables or adding calculated/summary columns, to improve read performance in scenarios where complex joins are too slow for reporting or high-traffic read operations.

Q58. What is BCNF (Boyce-Codd Normal Form)?

Answer: BCNF is a stricter version of 3NF that handles certain anomalies 3NF doesn't. A table is in BCNF if, for every functional dependency X -> Y, X must be a super key. It resolves cases where a table has multiple overlapping candidate keys.

Q59. What is an Entity-Relationship (ER) Diagram?

Answer: An ER Diagram is a visual representation of entities (tables), their attributes (columns), and the relationships between them (one-to-one, one-to-many, many-to-many), used during the database design phase before implementation.

Q60. How do you implement a many-to-many relationship in a relational database?

Answer: A many-to-many relationship is implemented using a junction/bridge/associative table that contains foreign keys referencing the primary keys of both related tables, effectively breaking the many-to-many relationship into two one-to-many relationships.

Q61. What is referential integrity?

Answer: Referential integrity is a property ensuring that relationships between tables remain consistent - specifically, that a foreign key value in a child table always corresponds to an existing primary key value in the parent table (or is NULL, if allowed), preventing orphaned records.

9. Advanced Queries & Window Functions

Q62. What are Window Functions and give some examples?

Answer: Window functions perform calculations across a set of rows related to the current row without collapsing them into a single output row (unlike GROUP BY). Common examples: ROW_NUMBER(), RANK(), DENSE_RANK(), LEAD(), LAG(), SUM() OVER(), and NTILE().

Q63. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Answer: ROW_NUMBER() assigns a unique sequential number to each row with no gaps, even for ties. RANK() assigns the same rank to ties but skips the next rank number(s) (e.g., 1,2,2,4). DENSE_RANK() assigns the same rank to ties but does not skip numbers (e.g., 1,2,2,3).

Q64. How would you find the second highest salary in a table?

Answer: Using a subquery: SELECT MAX(Salary) FROM Employees WHERE Salary < (SELECT MAX(Salary) FROM Employees). Alternatively, using window functions: SELECT Salary FROM (SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk FROM Employees) t WHERE rnk = 2.

Q65. What is a CTE (Common Table Expression) and why use it?

Answer: A CTE is a temporary, named result set defined using WITH ... AS () that exists only for the duration of a single query. It improves readability of complex queries, allows breaking logic into logical steps, and can be referenced multiple times within the same statement. It also enables recursive queries.

Q66. What is a Recursive CTE and when is it used?

Answer: A Recursive CTE references itself to process hierarchical or recursive data, such as organizational charts or bill-of-materials structures. It consists of an anchor member (base case) and a recursive member (which references the CTE itself), combined with UNION ALL.

Q67. What is the difference between a subquery and a JOIN?

Answer: A JOIN combines columns from two or more tables into one result set based on a related column, often more efficient for retrieving combined data. A subquery is a query nested inside another

query, often used to filter results or compute a scalar value, and can sometimes be less efficient than an equivalent JOIN, though this depends on the query optimizer.

Q68. What is PIVOT and UNPIVOT used for?

Answer: PIVOT rotates row data into columns (e.g., turning monthly sales rows into columns for each month), useful for creating summary reports. UNPIVOT does the reverse, converting columns back into rows. Both are supported natively in MSSQL; MySQL achieves similar results using conditional aggregation with CASE statements.

10. MSSQL vs MySQL – Platform-Specific Concepts

Q69. What are some key differences between MSSQL and MySQL?

Answer: MSSQL uses T-SQL, MySQL uses standard SQL with its own extensions. MSSQL uses IDENTITY for auto-increment, MySQL uses AUTO_INCREMENT. MSSQL uses TOP for limiting rows, MySQL uses LIMIT. MSSQL supports multiple storage engine features built-in, MySQL supports pluggable storage engines like InnoDB and MyISAM. MSSQL is commonly used with Windows/enterprise environments, MySQL is popular in open-source/web stacks.

Q70. How do you limit the number of rows returned in MSSQL vs MySQL?

Answer: In MSSQL: SELECT TOP 10 * FROM Employees; or using OFFSET-FETCH: SELECT * FROM Employees ORDER BY ID OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY. In MySQL: SELECT * FROM Employees LIMIT 10; and for pagination: LIMIT 10 OFFSET 20.

Q71. What are storage engines in MySQL? Name a few.

Answer: A storage engine determines how MySQL stores, retrieves, and manages data for a table. Common engines include InnoDB (default, supports transactions, foreign keys, row-level locking) and MyISAM (faster for read-heavy workloads but lacks transaction support and uses table-level locking).

Q72. What is the equivalent of MySQL's AUTO_INCREMENT in MSSQL?

Answer: MSSQL uses the IDENTITY property, e.g., ID INT IDENTITY(1,1) PRIMARY KEY, where the first value in parentheses is the seed and the second is the increment step.

Q73. How do you handle string concatenation differently in MySQL and MSSQL?

Answer: MySQL uses the CONCAT() function: SELECT CONCAT(FirstName, ' ', LastName). MSSQL supports the + operator for concatenation: SELECT FirstName + ' ' + LastName, and also supports CONCAT() as of SQL Server 2012+.

Q74. What is the equivalent of MySQL's IFNULL() in MSSQL?

Answer: MSSQL uses ISNULL(expression, replacement_value), which is functionally similar to MySQL's IFNULL(expression, replacement_value). Both platforms also support the ANSI-standard COALESCE() function, which accepts multiple arguments and returns the first non-NULL value.

Q75. What is SSMS and what MySQL tools are equivalent to it?

Answer: SSMS (SQL Server Management Studio) is Microsoft's GUI tool for managing SQL Server databases, running queries, and administration. Equivalent tools for MySQL include MySQL Workbench, phpMyAdmin, and DBeaver (which supports both platforms).

Q76. How does auto-commit behave differently between MySQL and MSSQL?

Answer: MySQL has auto-commit enabled by default, meaning each statement is committed immediately unless explicitly wrapped in START TRANSACTION. MSSQL, by default, also auto-commits each statement, but developers commonly use explicit BEGIN TRANSACTION / COMMIT / ROLLBACK blocks for multi-statement operations.

11. Backup, Recovery, Security & Administration

Q77. What are the different types of backups in MSSQL?

Answer: Full Backup captures the entire database at a point in time. Differential Backup captures only changes since the last full backup. Transaction Log Backup captures log records since the last log backup, enabling point-in-time recovery. Recovery models (Simple, Full, Bulk-Logged) determine how the transaction log is managed.

Q78. How do you take a backup in MySQL?

Answer: The most common tool is mysqldump, e.g., mysqldump -u root -p database_name > backup.sql, which exports the database as SQL statements. For larger databases, physical backup tools like MySQL Enterprise Backup or Percona XtraBackup are used for faster, non-blocking backups.

Q79. What is the difference between GRANT and REVOKE?

Answer: GRANT is used to give a user or role specific privileges on database objects (e.g., GRANT SELECT, INSERT ON Employees TO userA). REVOKE removes previously granted privileges from a user or role (e.g., REVOKE INSERT ON Employees FROM userA).

Q80. What is SQL Injection and how do you prevent it?

Answer: SQL Injection is a security vulnerability where an attacker manipulates a query by injecting malicious SQL through unsanitized user input. It's prevented by using parameterized queries/prepared statements instead of string concatenation, validating and sanitizing input, applying the principle of least privilege for database accounts, and using stored procedures with parameters.

Q81. What is the difference between authentication and authorization in a database context?

Answer: Authentication verifies who a user is (e.g., logging in with a username/password or Windows Authentication in MSSQL). Authorization determines what an authenticated user is allowed to do (e.g., which tables they can SELECT, INSERT, or modify), typically managed through GRANT/REVOKE and role membership.

Q82. What is data encryption and how is it applied in MSSQL/MySQL?

Answer: Data encryption protects sensitive data from unauthorized access. MSSQL offers Transparent Data Encryption (TDE) for encrypting data at rest, Always Encrypted for column-level encryption, and SSL/TLS for data in transit. MySQL supports data-at-rest encryption via InnoDB tablespace encryption and SSL/TLS connections for data in transit.

12. Performance Tuning & Query Optimization

Q83. How would you approach optimizing a slow-running query?

Answer: First, examine the execution plan (MSSQL) or run EXPLAIN (MySQL) to identify table scans, expensive operators, or missing indexes. Then check for missing/appropriate indexes, avoid SELECT *, rewrite correlated subqueries as joins where possible, ensure WHERE clause columns aren't wrapped in functions (which prevents index usage), update statistics, and check for parameter sniffing issues (MSSQL) or check the query cache/buffer pool sizing (MySQL).

Q84. What is a Query Execution Plan?

Answer: An Execution Plan is a visual or textual representation of the steps the database engine takes to execute a query - including which indexes are used, join algorithms chosen (nested loop, hash, merge), and estimated vs actual row counts - used to diagnose performance bottlenecks.

Q85. What causes a table scan and how do you avoid it?

Answer: A table scan happens when the query engine reads every row in a table, usually because there's no useful index for the WHERE/JOIN condition, the query uses a non-sargable expression (e.g., wrapping a column in a function), or the optimizer estimates a scan is cheaper for a large result set. It's avoided by creating appropriate indexes and writing sargable queries.

Q86. What does 'SARGable' mean in the context of SQL queries?

Answer: SARGable (Search ARGument-able) means a WHERE clause predicate can make use of an index for direct lookup. For example, WHERE YEAR(OrderDate) = 2024 is not SARGable because the function prevents index usage, whereas WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01' is SARGable.

Q87. What is the N+1 query problem and how do you avoid it?

Answer: The N+1 problem occurs when an application runs one query to fetch a list of records, then runs an additional query for each record to fetch related data, resulting in N+1 total queries. It's avoided by using JOINs, batch fetching (WHERE id IN (...)), or eager loading in ORMs instead of looping and querying individually.

Q88. What is connection pooling and why is it important?

Answer: Connection pooling reuses a pool of already-open database connections instead of opening and closing a new connection for every request, which reduces overhead and latency, improving application scalability and performance.

13. Error Handling & Programming Constructs

Q89. How do you implement error handling in MSSQL?

Answer: MSSQL uses TRY...CATCH blocks: BEGIN TRY -- code that might fail END TRY BEGIN CATCH -- handle error, e.g. SELECT ERROR_MESSAGE(), ERROR_NUMBER() END CATCH. Combined with transactions, you typically ROLLBACK inside the CATCH block if an error occurs.

Q90. How do you implement error handling in MySQL?

Answer: MySQL uses DECLARE ... HANDLER within stored procedures/functions, e.g., DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; -- error handling logic END;. It can also use signal conditions with SIGNAL SQLSTATE to raise custom errors.

Q91. What is the RAISERROR / THROW statement in MSSQL used for?

Answer: Both are used to generate custom error messages. THROW (introduced in SQL Server 2012) is simpler and re-throws the original error when used without parameters inside a CATCH block. RAISERROR is older, offers more formatting options (like severity levels) but doesn't preserve the original error state as cleanly as THROW.

Q92. What is a cursor and when would you use one?

Answer: A cursor allows row-by-row processing of a result set, useful when an operation cannot be set-based (e.g., calling an external procedure per row). Cursors are generally avoided when possible because set-based operations are far more performant; they should be a last resort.

Q93. What is the difference between a local temporary table and a global temporary table in MSSQL?

Answer: A local temporary table (#table) is visible only to the session that created it and is dropped when that session ends. A global temporary table (##table) is visible to all sessions and is dropped only when the creating session ends and no other session is referencing it.

14. Scenario-Based & Practical Questions

Q94. You have a query running slower after adding more data. How would you troubleshoot it?

Answer: I would check the execution plan to see if it's still using the expected indexes, verify that statistics are up to date (outdated statistics can cause the optimizer to choose a poor plan as data grows), check for index fragmentation, and consider whether new indexes are needed for the current data distribution and query patterns.

Q95. How would you migrate data from MySQL to MSSQL (or vice versa)?

Answer: I would use SQL Server Migration Assistant (SSMA) for MySQL-to-SQL Server migrations, which handles schema conversion, data migration, and object conversion. Alternatively, I could export data to a neutral format (CSV) and use SSIS (SQL Server Integration Services) or a custom ETL script, being careful about data type mapping differences (e.g., MySQL's TINYINT vs MSSQL's BIT) and syntax differences in stored procedures.

Q96. How do you handle versioning of database schema changes in a team project?

Answer: By using migration scripts managed under source control (e.g., using tools like Flyway, Liquibase, or Entity Framework Migrations), where each change is a numbered, incremental script applied in order, ensuring all environments (dev, QA, production) stay in sync.

Q97. Given a table of Employee(EmpID, Name, Salary, DeptID), write a query to get the highest paid employee in each department.

Answer: SELECT DeptID, Name, Salary FROM (SELECT DeptID, Name, Salary, RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rnk FROM Employee) t WHERE rnk = 1; This uses a window function partitioned by department to rank employees by salary within each department.

15. Role & Experience-Based Questions

Q98. As a database developer with about 1.3 years of experience, what kind of tasks have you typically handled?

Answer: Typical responsibilities at this stage include writing and optimizing SQL queries, creating and maintaining stored procedures/functions/views, designing simple to moderately complex table schemas, writing scripts for data migration/cleanup, assisting in performance tuning of slow queries, and supporting the application team with database-related bug fixes.

Q99. How do you stay updated with new features in MSSQL/MySQL?

Answer: By following official documentation and release notes (Microsoft Learn for SQL Server, MySQL's official docs), practicing on personal projects, reading community blogs and forums like Stack Overflow, and experimenting with new features such as window functions, JSON support, or new performance tools in a test environment.

Q100. How do you approach writing a complex query when you're not sure where to start?

Answer: I break the problem down into smaller logical steps - first identify the tables and relationships needed, write a basic query to pull the required base data, then progressively add filters, joins, grouping, or window functions, testing at each step using a small dataset before scaling to the full query. CTEs are often helpful to keep each step readable.